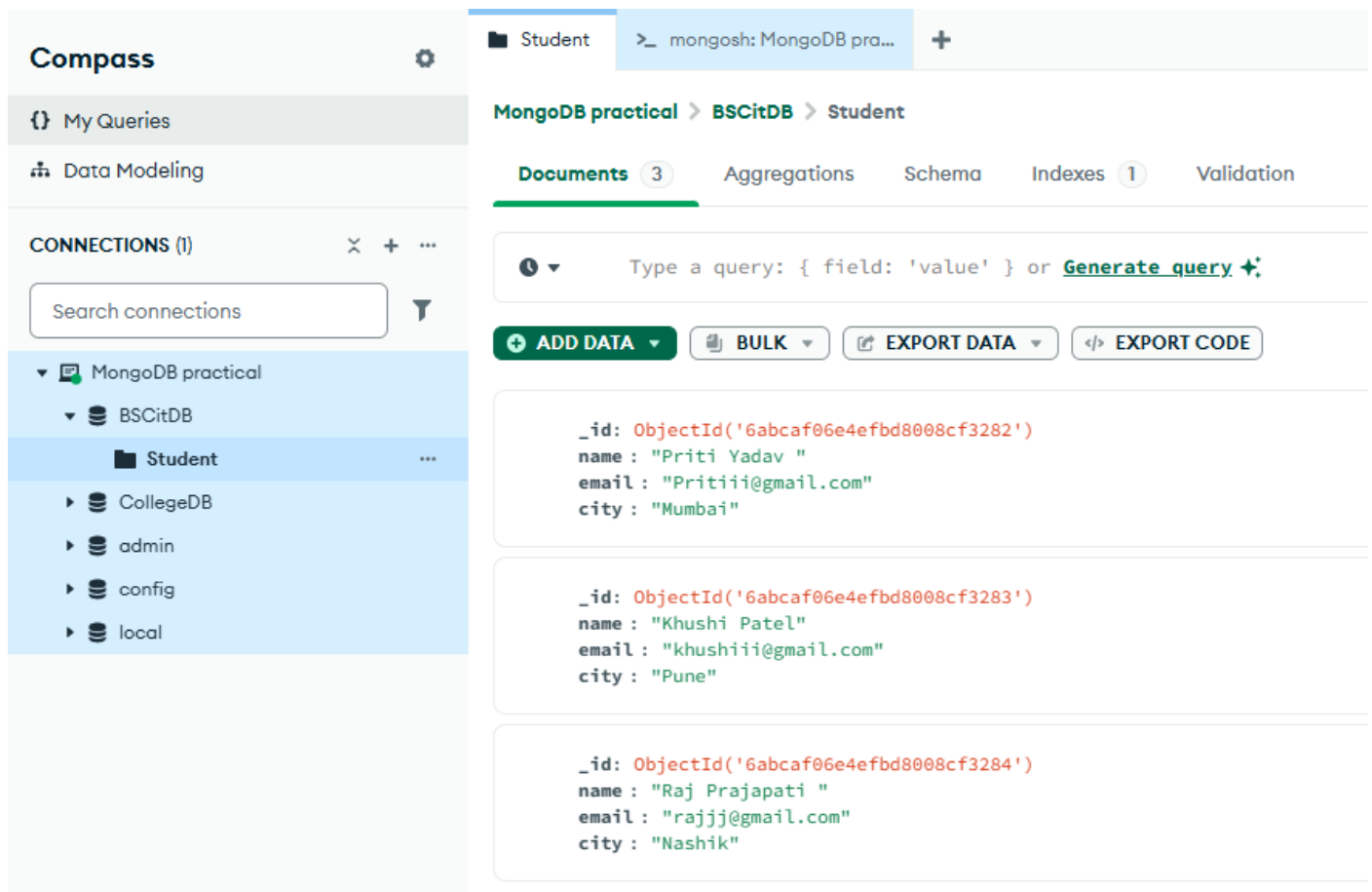


Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack Development Using Mern Stack
Practical – IX

Roll No.: T054	Name: Priti Yadav
Class: TYIT	Batch: 2
Date of Assignment: 30-09-26	Date/Time of Submission: 30-09-26

9. User Data Management

a. Create an API to add user details using Express and MongoDB.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows the 'MongoDB practical' connection with the 'BSCitDB' database selected. The 'Student' collection is highlighted. The main panel shows the 'Documents' tab for the 'Student' collection, displaying three documents:

```

_id: ObjectId('6abcaf06e4efbd8008cf3282')
name: "Priti Yadav "
email: "Pritiii@gmail.com"
city: "Mumbai"

_id: ObjectId('6abcaf06e4efbd8008cf3283')
name: "Khushi Patel"
email: "khushiii@gmail.com"
city: "Pune"

_id: ObjectId('6abcaf06e4efbd8008cf3284')
name: "Raj Prajapati "
email: "rajjj@gmail.com"
city: "Nashik"

```

App.jsx :

```

import { useEffect, useState } from "react";
import axios from "axios";
function App() {
  // Store all users
  const [users, setUsers] = useState([]);
  // Form data
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [city, setCity] = useState("");
  // Store ID of user being edited
  const [editId, setEditId] = useState(null);
  // Get Users from MongoDB

```

```

const getUsers = () => {
  axios.get("http://localhost:3000/users")
    .then((response) => {
      setUsers(response.data); })
    .catch((error) => {
      console.log(error);
    }); };

// Load users when page starts
useEffect(() => {getUsers();}, []);

// Add or Update User
const saveUser = () => { const user = {
  name: name,
  email: email,
  city: city
};

// UPDATE existing user
if (editId) {axios.put(`http://localhost:3000/users/${editId}`, user)
  .then((response) => {alert(response.data.message);
  clearForm();
  getUsers();
  })
  .catch((error) => {console.log(error);});
}

// ADD new user
else {axios.post("http://localhost:3000/users", user)
  .then((response) => {alert(response.data.message);
  clearForm();
  getUsers();
  })
  .catch((error) => {console.log(error);});
}
};

// Edit User
const editUser = (user) => {setEditId(user._id);
  setName(user.name);
  setEmail(user.email);
  setCity(user.city);
};

// Delete User
const deleteUser = (id) => {
  if (window.confirm("Are you sure you want to delete this user?")) {
    axios.delete(`http://localhost:3000/users/${id}`)
      .then((response) => {alert(response.data.message);
      getUsers();
      })
      .catch((error) => {console.log(error);});
  }
};

// Clear Form
const clearForm = () => {
  setName("");
  setEmail("");

```

```

setCity("");
setEditId(null); };
return (
<div>
<h1>User Management System</h1>
{/* User Form */}
<h2>
{editId ? "Update User" : "Add User"}
</h2>
<input type="text"
placeholder="Enter Name"
value={name}
onChange={(e) => setName(e.target.value)}
/>
<br /><br />
<input type="email"
placeholder="Enter Email"
value={email}
onChange={(e) => setEmail(e.target.value)}
/>
<br /><br />
<input type="text"
placeholder="Enter City"
value={city}
onChange={(e) => setCity(e.target.value)}
/>
<br /><br />
<button onClick={saveUser}>
{editId ? "Update User" : "Add User"}
</button>
{editId && (<button onClick={clearForm}>
Cancel
</button> )}
<hr />
{/* Display Users */}
<h2>User List</h2>
<table border="1">
<thead>
<tr>
<th>Name</th>
<th>Email</th>
<th>City</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{users.map((user) => (
<tr key={user._id}>
<td>{user.name}</td>
<td>{user.email}</td>

```

```

<td>{user.city}</td>
<td>
<button onClick={() => editUser(user)}>
Edit
</button>
<button onClick={() => deleteUser(user._id)}>
Delete
</button>
</td>
</tr>
))}
</tbody>
</table>
</div>
);}
export default App;

```

server.js :

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
// Middleware
app.use(cors());
app.use(express.json());
// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/UserDB")
.then(() => {
console.log("MongoDB Connected Successfully");
})
.catch((error) => {
console.log("MongoDB Connection Failed");
console.log(error);
});
// GET - Retrieve Users
app.get("/users", async (req, res) => {
try {
const users = await mongoose.connection.db
.collection("users")
.find()
.toArray();
res.json(users);
}
catch (error) {
res.status(500).json({message: "Error fetching users"});
}
});
// POST - Add User
app.post("/users", async (req, res) => {

```

```

try {
  const user = req.body;
  await mongoose.connection.db
    .collection("users")
    .insertOne(user);
  res.json({message: "User added successfully"}); }
catch (error) {
  res.status(500).json({message: "Error adding user"}); }
});

// PUT - Update User
app.put("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);
    const updatedUser = req.body;
    await mongoose.connection.db
      .collection("users")
      .updateOne({ _id: id },
        {
          $set: {
            name: updatedUser.name,
            email: updatedUser.email,
            city: updatedUser.city
          }
        }
      );
    res.json({message: "User updated successfully"});
  }
  catch (error) {
    res.status(500).json({message: "Error updating user"});
  }
});

// DELETE - Delete User
app.delete("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);
    await mongoose.connection.db
      .collection("users")
      .deleteOne({
        _id: id
      });
    res.json({message: "User deleted successfully"});
  }
  catch (error) {
    res.status(500).json({message: "Error deleting user"});
  }
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);

```

```
});
```

b. Design a React form to send user data to the backend.

```
{
  _id: ObjectId('6abcb0a7e9bf7fcc10469bfb'),
  name: 'Priti Yadav',
  email: 'Pritiii@gmail.com',
  city: 'Mumbai'
}
{
  _id: ObjectId('6abcb0a7e9bf7fcc10469bfd'),
  name: 'Raj Prajapati',
  email: 'raj@gmail.com',
  city: 'Nashik'
}
{
  _id: ObjectId('6abcc1bd2357c957d758b27f'),
  name: 'Aashwin ',
  email: 'ashwin@gmail.com',
  city: 'Delhi'
}
{
  _id: ObjectId('6abcc6e9e4efbd8008cf3293'),
  name: 'Ashwin',
  email: 'Ashwinh@gmail.com',
  city: 'Delhi'
}
```

c. Retrieve and display user data from MongoDB.

```
_id: ObjectId('6abcb0a7e9bf7fcc10469bfb')
name: "Priti Yadav"
email: "Pritiii@gmail.com"
city: "Mumbai"
```

```
_id: ObjectId('6abcb0a7e9bf7fcc10469bfd')
name: "Raj Prajapati"
email: "raj@gmail.com"
city: "Nashik"
```

```
_id: ObjectId('6abcc1bd2357c957d758b27f')
name: "Aashwin "
email: "ashwin@gmail.com"
city: "Delhi"
```

```
_id: ObjectId('6abcc6e9e4efbd8008cf3293')
name: "Ashwin"
email: "Ashwinh@gmail.com"
city: "Delhi"
```

d. Implement an update operation for existing records.

User Management System

Update User

User List

Name	Email	City	Actions
Priti Yadav	Pritiii@gmail.com	Mumbai	Edit Delete
Khushi Patel	khushiii@gmail.com	Pune	Edit Delete
Raj Prajapati	raj@gmail.com	Nashik	Edit Delete

User List

Name	Email	City	Actions
Priti Yadav	Pritiii@gmail.com	Mumbai	Edit Delete
Khushi Patel	khushiii@gmail.com	Santacruz	Edit Delete
Raj Prajapati	raj@gmail.com	Nashik	Edit Delete
Aashwin	ashwin@gmail.com	Delhi	Edit Delete

e. Implement the delete operation for selected records.

localhost:5173 says

Are you sure you want to delete this user?

User List

Name	Email	City	Actions
Priti Yadav	Pritiii@gmail.com	Mumbai	Edit Delete
Khushi Patel	khushiii@gmail.com	Santacruz	Edit Delete
Raj Prajapati	raj@gmail.com	Nashik	Edit Delete
Aashwin	ashwin@gmail.com	Delhi	Edit Delete

User List

Name	Email	City	Actions
Priti Yadav	Pritiii@gmail.com	Mumbai	Edit Delete
Raj Prajapati	raj@gmail.com	Nashik	Edit Delete
Aashwin	ashwin@gmail.com	Delhi	Edit Delete

f. Display updated records dynamically in React.

localhost:5173

User Management System

Update User

Raj Prajapati

raj@gmail.com

Mumbai

Update User Cancel

User List

Name	Email	City	Actions
Priti Yadav	Pritiii@gmail.com	Mumbai	Edit Delete
Raj Prajapati	raj@gmail.com	Nashik	Edit Delete
Aashwin	ashwin@gmail.com	Delhi	Edit Delete

```
_id: ObjectId('6abcb0a7e9bf7fcc10469bfd')
name : "Raj Prajapati"
email : "raj@gmail.com"
city : "Mumbai"
```